Nom: ANDRIANIRINA
Prénoms: Giovanie Andrica

CP5: METTRE EN PLACE UNE BASE DE DONNEES RELATIONNELLE

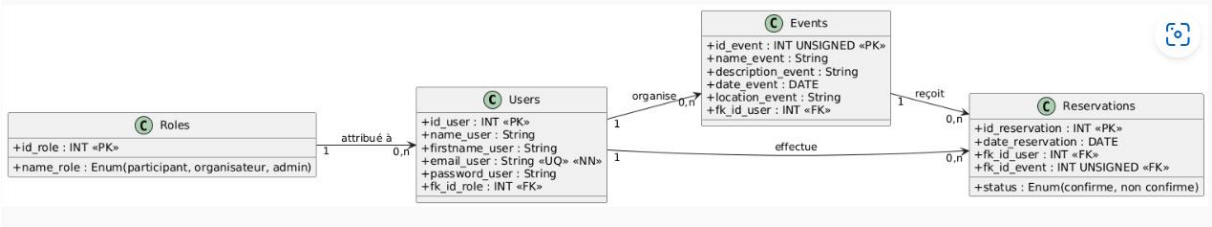
1- Description de la base de donnée

La base de données eventmanagement est conçue pour gérer des événements, Les utilisateurs qui les organisent ou y participent, ainsi que les réservations associées. Elle est composée de 4 tables reliées par des clés étrangères.

Cette application permettra à des utilisateurs de découvrir, de s'inscrire et de participer à divers événements tels que des conférences, des ateliers ou des rencontres
professionnelles.

.

2- Diagramme relationnel:



Le diagramme a été fait sur le site PlantText: PlantUML Editor Online - Free & Fast UML Diagram Tool | PlantText.
Elle a plusieurs avantages par rapport aux autres outils de création d'UML comme la facilité d'utilisation, la corrélation des textes avec les graphiques.Les diagrammes sont genérés à partir du texte , offrant une expérience de conception cohérente.
Dans le diagramme, il existe 4 table: table users, table pour reservations, table pour events et la table pour les roles(participant, organisateur ou administrateur).

Les tables sont reliées entre eux et a chacun leurs cardinalité.

```
Roles "1" --> "0,n" Users : attribué à
Users "1" --> "0,n" Events : organise
Users "1" --> "0,n" Reservations : effectue
Events "1" --> "0,n" Reservations : reçoit
@enduml
```

3- Justification des choix de conception

Chaque table représente une entité métier distincte afin d'éviter la redondance et les anomalies de mise à jour.
La table roles est séparée de users pour permettre une gestion centralisée des droits et faciliter l'ajout de nouveaux rôles sans modifier la structure de la table utilisateurs.

- Choix de type de donnée:

Le choix de type de donnée dépend des données qu'on veut transmettre.

INT AUTO_INCREMENT: Identifiant numérique simple, performant pour les jointures et indexation.

VARCHAR(100):longueur suffisante pour tout email valide.
ENUM(...): Liste fixe de valeurs connues

DATE: Stockage de la date uniquement (pas d'heure nécessaire pour un événement).
INT:Valeur toujours positive

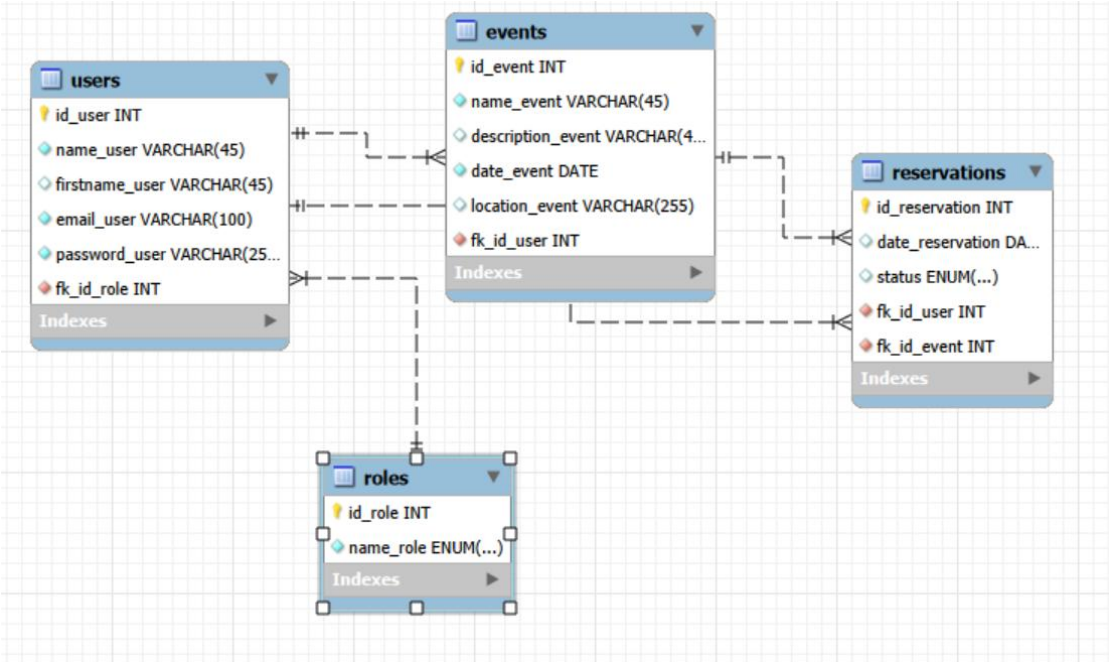
- Contraintes

PRIMARY KEY sur chaque table : Identifie chaque enregistrement de manière unique.
FOREIGN KEY avec ON DELETE NO ACTION : Protège l'intégrité référentielle : impossible de supprimer un parent s'il a des enfants.
UNIQUE sur email_user : Empêche deux comptes avec le même email.
NOT NULL: sur les colonnes essentielles : Garantit que les données obligatoires sont toujours présentes.
INDEX sur les clés étrangères : Accélère les jointures et les recherches filtrées.

4- Présentation des requetes SQL

Les requêtes ci-dessous illustrent les principales User Stories par la base de données eventmanagement. Les requettes ont été par phpmyAdmin.

PhpMyAdmin est une interface graphique accessible depuis un navigateur web qui facilite l'administration des bases de données MySQL et MariaDB sans nécessiter de connaissances approfondies en SQL. Mais avant de faire la requete , j'ai d'abord utilisé MySQL WorkBench les tables et je les ai importé sur phpMyAdmin.



Chacune des requetes s’adapte aux user Stories .

=>
```sql
-- Use1 créer un compte utilisateur --
-- ici la personne peut créer son nom d'utilisateur en inserant son nom, prénom, son email et le mot de passe --
INSERT INTO users (name_user, firstname_user, email_user, password_user)
VALUES ('Jannie', 'Pont', 'jn@mail.com', 'jn45678');

-- Use2 Consulter les événements disponibles --
SELECT *
FROM events;
```

```sql
-- Use3 S'inscrire à un événement --
INSERT INTO reservations (date_reservation, status, fk_id_user, fk_id_event)
VALUES (CURDATE(), 'confirme', 2, 6);
```

```sql
-- Use4 Consulter mes inscriptions personnelles --
SELECT e.name_event, e.date_event
FROM reservations r
JOIN events e ON r.fk_id_event = e.id_event
WHERE r.fk_id_user = 2;
```

```sql
-- Use5  Créer un nouvel événement --
INSERT INTO events (name_event, description_event, date_event, location_event, fk_id_user)
VALUES ('Conférence IA', 'Découverte de l''IA', '2026-09-15', 'Paris', 1);
```

```sql
-- US7 : Analyser la popularité des événements --
SELECT e.name_event, COUNT(r.id_reservation) AS nombre_participants
```

```sql
FROM events e
LEFT JOIN reservations r ON e.id_event = r.fk_id_event
GROUP BY e.id_event, e.name_event
ORDER BY nombre_participants DESC;
```

```sql
-- US8 : Lister tous les utilisateurs enregistrés --
SELECT id_user, name_user, firstname_user, email_user
FROM users
ORDER BY name_user ASC;
```

```sql
-- US9 : Mettre à jour les informations d'un événement --
-- (exemple : modifier le lieu et la description de l'événement id=6)
UPDATE events
SET location_event = 'Ginasservis',
    description_event = 'Description corrigée et mise à jour'
WHERE id_event = 6
  AND fk_id_user = 1; -- sécurité : seul l'organisateur propriétaire peut modifier
```

```sql
-- US10 : Annuler son inscription à un événement --
-- (exemple : l'utilisateur id=2 annule sa réservation pour l'événement id=6)
DELETE FROM reservations
WHERE fk_id_user = 2
  AND fk_id_event = 6;
```

```sql
-- US11 : Supprimer un événement annulé --
-- Étape 1 : supprimer d'abord les réservations liées (respect des clés étrangères)
DELETE FROM reservations
WHERE fk_id_event = 6;
```

```sql
-- Étape 2 : supprimer l'événement
DELETE FROM events
WHERE id_event = 6
  AND fk_id_user = 1; --  seul l'organisateur propriétaire peut supprimer
```